

Prototype, evaluate, *price*.

The area where AI changed my product management most is not writing: it is **collapsing the distance between a product decision and working software**. I stopped writing documents that ask engineering to go find out, and started answering the question myself before the sprint begins. At PixelBin, a suite of 15-plus AI media tools whose P&L I owned, that shift took release cadence from two or three a year to ten-plus in a single quarter.

THE METHOD

MOVE 01 Prototype instead of specify When a decision needs evidence, I build the thing. With Claude and Codex a working prototype takes a day or two, goes in front of users or the team, and what happens next decides the roadmap. The prototype is the spec.	MOVE 02 Evaluate instead of argue Model and feature choices get settled by a scored evaluation set built on our own prompts and user data, never a vendor benchmark or a persuasive demo. A claim about quality is a hypothesis until it has a number.	MOVE 03 Price from measured cost Generation economics move weekly, so pricing follows measured cost per accepted output rather than per call, and the credit ladder is held to it. Quality, latency and unit cost are one decision, not three.
---	--	--

WHAT IT PRODUCED AT PIXELBIN

Release cadence	10+ per quarter	Against two or three in the prior year, after resetting product strategy, ICP and roadmap against real team capacity.
Revenue	3 months	Consecutive months at the highest revenue the suite had recorded, after aligning engineering, ML, growth and SEO KPIs to per-tool revenue goals.
Reliability	Zero	Customer complaints on the main generation tools through that period of rapid shipping, carried by error dashboards and release governance.
Distribution	New channel	Opened MCP and a public Claude skill as acquisition surfaces, so agents call the tools directly and the product reaches users who never open the website.

The part that is easy to miss: shipping faster is the visible result. The durable one is that decisions became reviewable. A model choice, a price change or a launch call each leaves behind an evaluation set, a cost model and a trace, so a successor can audit the reasoning instead of inheriting my judgement.

WHERE THE LEVERAGE ACTUALLY COMES FROM

- **Tooling built for the team, not just for me.** Growth dashboards over PostHog, GA4, Paddle and Sentry, published with the exact query visible so anyone can check the number rather than trust it.
- **Skills, not prompts.** Twenty-plus custom Claude skills with deploy-ready outputs, so recurring product work runs as a repeatable process. Several are public and in use by other people.
- **Instrumentation before intuition.** Prompt types, user profiles and model or feature usage mined directly, so feature priority comes from observed behaviour, not a data request that takes a week.
- **Architecture close enough to be useful.** Working with engineering on GCP on batch sizing, pipelines and model-performance dashboards, which is what keeps prototypes buildable rather than decorative.

One decision, end to end

Worked example · which model powers a feature, and what a credit should cost

Anand Pareek
AI Product Leader

A recurring decision in an AI media suite, and the one I ran the method on most often, across image and video generation with models including Nano Banana, Kling, PixVerse and Seedance. **Getting it wrong is expensive in two directions at once.** Use the strongest model everywhere and the margin disappears. Use the cheapest and users churn on quality they can see.

HOW THE DECISION RAN

- 1 Define acceptance before looking at any model.** What does a good output mean for this tool and this user, who is a non-designer rather than a professional creator? Written down first, so the bar cannot quietly move to fit a result.
- 2 Score on our own data, not the vendor's.** An evaluation set built from real user prompts across the tools and scored against that written bar. A separate pipeline and dashboard, built with engineering on GCP, did the same job for watermark-removal model performance.
- 3 Measure cost and latency per accepted output, not per call.** Failed and retried generations are part of the cost of a result the user actually keeps, and that is the number the business pays.
- 4 Set the I/O parameters and UX around the winner.** Which controls to expose, which to hide behind good defaults, and what the product promises, so quality becomes a product decision rather than a model property.
- 5 Push the result into pricing.** The credit ladder follows measured cost per accepted output, under the rule that per-credit price must fall at every tier. When model economics move, the ladder is rebuilt, not left until the annual review.

What this replaces: the common version is a bake-off judged by eye, then a price copied from a competitor's page. That is faster for one week and unmanageable by the third model. This version is slower once and then reusable, because the evaluation set and the cost model outlive any single model choice.

PUBLIC ARTEFACTS YOU CAN INSPECT

PixelBin Claude Skill 14 stars · MIT	Turns the product into a full media pipeline inside a Claude conversation: 85-plus AI APIs and 60-plus chainable transformations. Drop fifty product photos in chat and get back e-commerce-ready images in one prompt. Distribution as a product decision, shipped publicly.	github.com/anandpareek-hub/pixelbin-claude-skill
Claude Skills Toolkit 5 skills	Product analytics on PostHog, revenue on Paddle, SEO on Search Console, reporting and media. Each auto-discovers the account's own schema, so the same skill works for a different business without a rewrite.	github.com/anandpareek-hub/claude-skills-toolkit
power-coding 6 stars	An operating discipline for building with LLM coding agents: session handoff, evaluation loops, failure-mode scans and a pre-commit secret gate. Plain markdown, no daemons, works across Claude Code, Codex and Gemini CLI.	github.com/APareek89/power-coding
agentic-learning-skill Accepted upstream	Turns any topic into a self-contained interactive lesson, or a brief into an installable agent skill. Accepted by the maintainer into the community awesome-Claude-skills list.	github.com/APareek89/agentic-learning-skill

Why the method transfers: define acceptance, score on your own data, measure cost per accepted result, price from it. Media generation is where I ran it; nothing in it is specific to media. It also survives the model cycle, which matters now more than it used to. Every frontier release resets the quality and price landscape, and a team that owns its evaluation set re-runs it in a day while a team that does not starts the argument over.